ORIENTATION TO COMPUTING

UNIT 3: OPERATING SYSTEM ARCHITECTURE, PROCESS MANAGEMENT & MEMORY SYSTEMS
Curriculum: Lovely Professional University | School of Computer Science & Engineering
Reference: Operating System Concepts by Silberschatz, Galvin & Gagne

>> 3.1 Role, Architecture and Evolutionary Types of Operating Systems

An Operating System (OS) is core system software that acts as an intermediary between computer hardware and user application programs, managing hardware resources efficiently:
* Core Objectives: Resource abstraction (hiding hardware complexity), Resource allocation (CPU, memory, storage, I/O devices), and Protection/Security.
* Evolutionary Paradigms: Batch Systems (jobs grouped without interaction), Multiprogramming (keeping CPU busy by running another job when one waits for I/O), Time-Sharing (multitasking via rapid CPU context switching), Distributed Systems, and Real-Time Operating Systems (RTOS - strict deadline guarantees: Hard RTOS vs Soft RTOS).
* Dual-Mode Operation and Hardware Protection:
- User Mode (Mode bit = 1): Application software executes with restricted privileges; cannot execute privileged machine instructions or access hardware registers directly.
- Kernel Mode / Supervisor Mode (Mode bit = 0): Unrestricted hardware access. Traps and System Calls transition CPU from User Mode to Kernel Mode.
* OS Structural Models: Monolithic Kernel (Linux - all OS services reside in kernel space; maximum performance) vs Microkernel (Mach, QNX - only IPC, basic scheduling, and virtual memory in kernel; file systems and drivers run as user space daemons; high reliability and fault isolation).

>> 3.2 The Process Model and Lifecycle Transitions

* Process Definition: An active program in execution (including Program Counter, CPU registers, Stack, Heap, Data, and Text sections).
* Process Control Block (PCB): The data structure maintaining process state: Process ID (PID), Program Counter, CPU Registers, CPU Scheduling Information, Memory-Management Info, I/O Status & Open File Descriptors.
* 5-State Process Model: New -> Ready (in memory waiting for CPU) -> Running (instructions executing) -> Waiting/Blocked (waiting for I/O or event) -> Terminated.
* Context Switching: The mechanism of saving the state of the currently running process to its PCB and loading the saved state of another ready process into CPU

registers. Pure computational overhead.

>> 3.3 CPU Scheduling Algorithms and Metrics

* Optimization Goals: Maximize CPU Utilization and Throughput; Minimize Turnaround Time (TAT = Completion - Arrival), Waiting Time (WT = TAT - Burst Time), and Response Time.
* Core Scheduling Algorithms:
- 1. First-Come First-Served (FCFS): Non-preemptive. Suffers from Convoy Effect (short processes stuck behind a long CPU-bound process).
- 2. Shortest Job First (SJF) / Shortest Remaining Time First (SRTF): Provably optimal average waiting time. Preemptive variant is SRTF. Suffers from potential starvation of long processes.
- 3. Round Robin (RR): Preemptive. Each process gets a fixed time quantum q. If burst > q, process is preempted and placed at tail of ready queue. Fair, interactive.
- 4. Priority Scheduling: CPU assigned to highest priority process. Starvation prevented via Aging (gradually increasing priority of waiting processes).

>> 3.4 Inter-Process Synchronization and Deadlock Theory

* Race Condition: A concurrency flaw where output depends critically on the non-deterministic execution sequence or interleaving of competing threads.
* Critical Section Problem: Code segment accessing shared resources. Solutions must satisfy: Mutual Exclusion, Progress, and Bounded Waiting.
* Semaphores: Synchronization primitive with atomic operations wait() (P) and signal() (V). Binary Semaphore (Mutex) takes values 0 and 1; Counting Semaphore manages multiple resource instances.
* Deadlock: A permanent blocking condition where every process in a set waits for a resource held by another process in the set.
* The 4 Coffman Necessary Conditions for Deadlock:
- 1. Mutual Exclusion: At least one resource held in non-shareable mode.
- 2. Hold and Wait: Process holding resources requests additional resources.
- 3. No Preemption: Resources cannot be forcibly confiscated from a process.
- 4. Circular Wait: A closed loop chain of processes where P_i waits for resource held by P_(i+1).
* Deadlock Handling: Deadlock Prevention, Deadlock Avoidance (Banker''s Algorithm for safe states), Deadlock Detection and Recovery.

>> 3.5 Virtual Memory, Paging and Page Replacement Algorithms

* Physical vs Logical Address: Logical address generated by CPU; translated into Physical Address in DRAM by Memory Management Unit (MMU).
* Paging Architecture: Logical memory divided into fixed-size blocks called Pages; physical memory divided into equal-sized blocks called Frames. Page Table maps Page Number to Frame Number.
* Translation Lookaside Buffer (TLB): High-speed associative hardware cache storing recently used Page-to-Frame mappings to accelerate address translation.
* Virtual Memory: Allows execution of processes not completely loaded in physical RAM using Demand Paging.
* Page Replacement Algorithms (executed upon Page Fault when all RAM frames are occupied):
- FIFO (First-In, First-Out): Suffers from Belady''s Anomaly (more frames can produce more page faults).
- LRU (Least Recently Used): Replaces page that has not been referenced for longest duration. Optimal practical algorithm.
- Optimal (OPT): Replaces page that will not be used for longest future duration (theoretical benchmark).

[PROBLEM 1: Comparative CPU Scheduling Analysis: FCFS vs Round Robin]

Problem Statement: Three processes arrive at time 0 with the following CPU burst times: P1 = 24 ms, P2 = 3 ms, P3 = 3 ms. Compare First-Come First-Served (FCFS) and Round Robin (RR with time quantum q = 4 ms) in terms of: (1) Gantt Chart, (2) Turnaround Time, and (3) Average Waiting Time.
Mathematical Derivation & Evaluation:
* 1. FCFS Scheduling Execution:
- Gantt Chart: [P1: 0 to 24] [P2: 24 to 27] [P3: 27 to 30].
- Completion Times: P1 = 24, P2 = 27, P3 = 30.
- Waiting Times: P1 = 0 ms, P2 = 24 ms, P3 = 27 ms.
- Average Waiting Time (FCFS) = (0 + 24 + 27) / 3 = 51 / 3 = 17.0 ms.
* 2. Round Robin Scheduling Execution (q = 4 ms):
- Gantt Chart: [P1: 0 to 4] [P2: 4 to 7] [P3: 7 to 10] [P1: 10 to 30].
- Note: P2 finishes at t = 7; P3 finishes at t = 10; P1 resumes at t = 10 and finishes remaining 20 ms at t = 30.
- Completion Times: P1 = 30, P2 = 7, P3 = 10.
- Waiting Times: P1 = (0) + (10 - 4) = 6 ms; P2 = 4 ms; P3 = 7 ms.
- Average Waiting Time (RR) = (6 + 4 + 7) / 3 = 17 / 3 = 5.67 ms.
* 3. Engineering Conclusion: Round Robin drastically slashes average waiting time from 17.0 ms to 5.67 ms (a 3x improvement) by eliminating the convoy effect.

[PROBLEM 2: Deadlock Avoidance using Dijkstra''s Banker''s Algorithm]

Problem Statement: A system has 5 processes (P0 to P4) and 3 resource types A (10 instances), B (5 instances), C (7 instances). Current Allocation and Max Need matrices are given:

* Allocation: P0=(0,1,0), P1=(2,0,0), P2=(3,0,2), P3=(2,1,1), P4=(0,0,2). Sum allocated = (7, 2, 5).
* Max Matrix: P0=(7,5,3), P1=(3,2,2), P2=(9,0,2), P3=(2,2,2), P4=(4,3,3).
* Calculate: Available Vector and verify if the system is in a Safe State by finding the Safe Sequence.

Mathematical Execution:

* 1. Available Vector = Total - Allocated = (10,5,7) - (7,2,5) = (3, 3, 2).
* 2. Calculate Need Matrix (Need = Max - Allocation):
- Need P0 = (7,5,3) - (0,1,0) = (7, 4, 3)
- Need P1 = (3,2,2) - (2,0,0) = (1, 2, 2)
- Need P2 = (9,0,2) - (3,0,2) = (6, 0, 0)
- Need P3 = (2,2,2) - (2,1,1) = (0, 1, 1)
- Need P4 = (4,3,3) - (0,0,2) = (4, 3, 1)
* 3. Safety Algorithm Iterations:
- Iteration 1: Check P1: Need (1,2,2) <= Available (3,3,2) -> TRUE! P1 executes, releases allocation. New Available = (3,3,2) + (2,0,0) = (5, 3, 2).
- Iteration 2: Check P3: Need (0,1,1) <= Available (5,3,2) -> TRUE! P3 executes, releases allocation. New Available = (5,3,2) + (2,1,1) = (7, 4, 3).
- Iteration 3: Check P0: Need (7,4,3) <= Available (7,4,3) -> TRUE! P0 executes, releases allocation. New Available = (7,4,3) + (0,1,0) = (7, 5, 3).
- Iteration 4: Check P2: Need (6,0,0) <= Available (7,5,3) -> TRUE! P2 executes, releases allocation. New Available = (7,5,3) + (3,0,2) = (10, 5, 5).
- Iteration 5: Check P4: Need (4,3,1) <= Available (10,5,5) -> TRUE! P4 executes. New Available = (10, 5, 7).
* 4. Safe Sequence Found: < P1, P3, P0, P2, P4 >. The system is provably DEADLOCK FREE.

[PROBLEM 3: Virtual Memory Page Replacement: FIFO vs LRU Analysis]

Problem Statement: Consider the following memory page reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2. For a physical memory allocated 3 page frames (initially empty), compute the number of page faults for: (1) FIFO, and (2) LRU.

Step-by-Step Fault Trace:
* 1. FIFO (First-In First-Out) Trace (3 frames):
- Ref 7: [7, -, -] (Fault 1) | Ref 0: [7, 0, -] (Fault 2) | Ref 1: [7, 0, 1] (Fault 3)
- Ref 2: Replace 7 -> [2, 0, 1] (Fault 4) | Ref 0: Hit [2, 0, 1]
- Ref 3: Replace 0 -> [2, 3, 1] (Fault 5) | Ref 0: Replace 1 -> [2, 3, 0] (Fault 6)
- Ref 4: Replace 2 -> [4, 3, 0] (Fault 7) | Ref 2: Replace 3 -> [4, 2, 0] (Fault 8)
- Ref 3: Replace 0 -> [4, 2, 3] (Fault 9) | Ref 0: Replace 4 -> [0, 2, 3] (Fault 10)
- Ref 3: Hit [0, 2, 3] | Ref 2: Hit [0, 2, 3].
- Total FIFO Page Faults = 10.
* 2. LRU (Least Recently Used) Trace (3 frames):
- Ref 7, 0, 1: [7, 0, 1] (Faults 1, 2, 3)
- Ref 2: Replace least recently used (7) -> [2, 0, 1] (Fault 4)
- Ref 0: Hit -> Recent order: [1, 2, 0]
- Ref 3: Replace 1 -> [2, 0, 3] (Fault 5) | Ref 0: Hit -> Recent: [2, 3, 0]
- Ref 4: Replace 2 -> [4, 3, 0] (Fault 6) | Ref 2: Replace 3 -> [4, 0, 2] (Fault 7)
- Ref 3: Replace 4 -> [3, 0, 2] (Fault 8) | Ref 0: Hit [3, 2, 0] | Ref 3: Hit [2, 0, 3]
| Ref 2: Hit [0, 3, 2].
- Total LRU Page Faults = 8.
* 3. Conclusion: LRU achieves lower page fault rate (8 vs 10) by exploiting temporal locality of reference.

[PROBLEM 4: Logical-to-Physical Address Translation in Paging]

Problem Statement: A 32-bit computing system uses paging with 4 KB (2^12 bytes) page size. A process generates logical address 0x0002A3F4. If Page Table entry for Page 0x0002A holds Frame Number 0x0010B, determine the exact physical DRAM address.
Mathematical Derivation:
* 1. Address Bit Division: Page Size = 4 KB = 2^12 bytes -> Offset occupies 12 bits (3 hex digits). Remaining 20 bits (5 hex digits) represent Page Number.
* 2. Decompose Logical Address 0x0002A3F4:
- Page Number (p) = 0x0002A.
- Offset (d) = 0x3F4.
* 3. Frame Lookup: Page Table indicates Frame Number (f) = 0x0010B.
* 4. Form Physical Address: Physical Address = (Frame Number << 12) | Offset = 0x0010B000 + 0x3F4 = 0x0010B3F4.